

Методы экземпляра класса

Класс Circle

Реализуйте класс `Circle`, описывающий круг. При создании экземпляра класс должен принимать один аргумент:

- `radius` — радиус круга

Экземпляр класса `Circle` должен иметь три атрибута:

- `radius` — радиус круга
- `diameter` — диаметр круга
- `area` — площадь круга

Примечание 1. Площадь круга вычисляется по формуле πr^2 , где r — радиус круга, π — константа, которая выражает отношение длины окружности к ее диаметру.

Примечание 2. Импортировать константу π можно из модуля `math`:

```
from math import pi
```

Примечание 3. Дополнительная проверка данных на корректность не требуется. Гарантируется, что реализованный класс используется только с корректными данными.

Класс Gun

Реализуйте класс `Gun`, описывающий ружье. При создании экземпляра класс не должен принимать никаких аргументов.

Класс `Gun` должен иметь один метод экземпляра:

- `shoot()` — метод, при первом вызове которого выводится строка `pif`, при втором — `paf`, при третьем — `pif`, при четвертом — `paf`, и так далее

Класс Numbers

Реализуйте класс `Numbers`, описывающий изначально пустой расширяемый набор целых чисел. При создании экземпляра класс не должен принимать никаких аргументов.

Класс `Numbers` должен иметь три метода экземпляра:

- `add_number()` — метод, принимающий в качестве аргумента целое число и добавляющий его в набор
- `get_even()` — метод, возвращающий список всех четных чисел из набора
- `get_odd()` — метод, возвращающий список всех нечетных чисел из набора

Примечание 1. Числа в списках, возвращаемых методами `get_even()` и `get_odd()`, должны располагаться в том порядке, в котором они были добавлены в набор.

Примечание 2. Дополнительная проверка данных на корректность не требуется. Гарантируется, что реализованный класс используется только с корректными данными.

Класс TextHandler

Будем называть словом любую последовательность из одной или более букв. Текстом будем считать набор слов, разделенных пробельными символами.

Реализуйте класс `TextHandler`, описывающий изначально пустой расширяемый набор слов. При создании экземпляра класс не должен принимать никаких аргументов.

Экземпляр класса `TextHandler` должен иметь три метода:

- `add_words()` — метод, принимающий в качестве аргумента текст и добавляющий слова из данного текста в набор
- `get_shortest_words()` — метод, возвращающий актуальный список самых коротких слов в наборе
- `get_longest_words()` — метод, возвращающий актуальный список самых длинных слов в наборе

Примечание 1. Слова в списках, возвращаемых методами `get_shortest_words()` и `get_longest_words()`, должны располагаться в том порядке, в котором они были добавлены в набор.

Примечание 2. Дополнительная проверка данных на корректность не требуется. Гарантируется, что реализованный класс используется только с корректными данными.

Класс Knight 🐎

Реализуйте класс `Knight`, описывающий шахматного коня. При создании экземпляра класс должен принимать три аргумента в следующем порядке:

- `horizontal` — координата коня по горизонтальной оси, представленная латинской буквой от `a` до `h`
- `vertical` — координата коня по вертикальной оси, представленная целым числом от `1` до `8` включительно
- `color` — цвет коня (`black` или `white`)

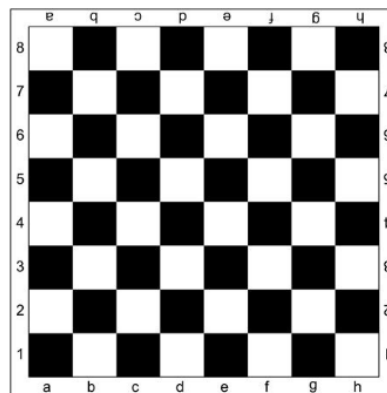
Экземпляр класса `Knight` должен иметь три атрибута:

- `horizontal` — координата коня на шахматной доске по горизонтальной оси
- `vertical` — координата коня на шахматной доске по вертикальной оси
- `color` — цвет коня

Класс `Knight` должен иметь четыре метода экземпляра:

- `get_char()` — метод, возвращающий символ `N`
- `can_move()` — метод, принимающий в качестве аргументов координаты клетки по горизонтальной и по вертикальной осям и возвращающий `True`, если конь может переместиться на клетку с данными координатами, или `False` в противном случае
- `move_to()` — метод, принимающий в качестве аргументов координаты клетки по горизонтальной и по вертикальной осям и заменяющий текущие координаты коня на переданные. Если конь из текущей клетки не может переместиться на клетку с указанными координатами, его координаты должны остаться неизменными
- `draw_board()` — метод, печатающий шахматное поле, отмечающий на этом поле коня и клетки, на которые может переместиться конь. Пустые клетки должны быть отображены символом `.`, конь — символом `N`, клетки, на которые может переместиться конь, — символом `*`

Примечание 1. Шахматное поле имеет вид:



Примечание 2. Дополнительная проверка данных на корректность не требуется. Гарантируется, что реализованный класс используется только с корректными данными.

Модификаторы доступа и аксессоры

Класс Circle

Реализуйте класс `Circle`, описывающий круг. При создании экземпляра класс должен принимать один аргумент:

- `radius` — радиус круга

Экземпляр класса `Circle` должен иметь три атрибута:

- `_radius` — радиус круга
- `_diameter` — диаметр круга
- `_area` — площадь круга

Класс `Circle` должен иметь три метода экземпляра:

- `get_radius()` — метод, возвращающий радиус круга
- `get_diameter()` — метод, возвращающий диаметр круга
- `get_area()` — метод, возвращающий площадь круга

Примечание 1. Площадь круга вычисляется по формуле πr^2 , где r — радиус круга, π — константа, которая выражает отношение длины окружности к ее диаметру.

Примечание 2. Импортировать константу π можно из модуля `math`:

```
from math import pi
```

Примечание 3. Дополнительная проверка данных на корректность не требуется. Гарантируется, что реализованный класс используется только с корректными данными.

Sample Input 1:

```
circle = Circle(1)

print(circle.get_radius())
print(circle.get_diameter())
print(round(circle.get_area()))
```

Sample Output 1:

```
1
2
3
```

Класс BankAccount

Реализуйте класс `BankAccount`, описывающий банковский счет. При создании экземпляра класс должен принимать один аргумент:

- `balance` — баланс счета, по умолчанию имеет значение `0`

Экземпляр класса `BankAccount` должен иметь один атрибут:

- `_balance` — баланс счета

Класс `BankAccount` должен иметь четыре метода экземпляра:

- `get_balance()` — метод, возвращающий актуальный баланс счета
- `deposit()` — метод, принимающий в качестве аргумента число `amount` и увеличивающий баланс счета на `amount`
- `withdraw()` — метод, принимающий в качестве аргумента число `amount` и уменьшающий баланс счета на `amount`. Если `amount` превышает количество средств на балансе счета, должно быть возбуждено исключение `ValueError` с сообщением:

```
На счете недостаточно средств
```

- `transfer()` — метод, принимающий в качестве аргументов банковский счет `account` и число `amount`. Метод должен уменьшать баланс текущего счета на `amount` и увеличивать баланс счета `account` на `amount`. Если `amount` превышает количество средств на балансе текущего счета, должно быть возбуждено исключение `ValueError` с сообщением:

```
На счете недостаточно средств
```

Примечание 1. Числами будем считать экземпляры классов `int` и `float`.

Примечание 2. Дополнительная проверка данных на корректность не требуется. Гарантируется, что реализованный класс используется только с корректными данными.

Sample Input 1:

```
account = BankAccount()

print(account.get_balance())
account.deposit(100)
print(account.get_balance())
account.withdraw(50)
print(account.get_balance())
```

Sample Output 1:

```
0
100
50
```

Sample Input 2:

```
account = BankAccount(100)

try:
    account.withdraw(150)
except ValueError as e:
    print(e)
```

Sample Output 2:

```
На счете недостаточно средств
```

Класс User

Реализуйте класс `User`, описывающий интернет-пользователя. При создании экземпляра класс должен принимать два аргумента в следующем порядке:

- `name` — имя пользователя. Если `name` не является непустой строкой, состоящей только из букв, должно быть возбуждено исключение `ValueError` с текстом:

```
Некорректное имя
```

- `age` — возраст пользователя. Если `age` не является целым числом, принадлежащим отрезку `[0; 110]`, должно быть возбуждено исключение `ValueError` с текстом:

```
Некорректный возраст
```

Экземпляр класса `User` должен иметь два атрибута:

- `_name` — имя пользователя
- `_age` — возраст пользователя

Класс `User` должен иметь четыре метода экземпляра:

- `get_name()` — метод, возвращающий имя пользователя
- `set_name()` — метод, принимающий в качестве аргумента значение `new_name` и изменяющий имя пользователя на `new_name`. Если `new_name` не является непустой строкой, состоящей только из букв, должно быть возбуждено исключение `ValueError` с текстом:

```
Некорректное имя
```

- `get_age()` — метод, возвращающий возраст пользователя
- `set_age()` — метод, принимающий в качестве аргумента значение `new_age` и изменяющий возраст пользователя на `new_age`. Если `new_age` не является целым числом, принадлежащим отрезку `[0; 110]`, должно быть возбуждено исключение `ValueError` с текстом:

```
Некорректный возраст
```

Примечание 1. Если при создании экземпляра класса `User` имя и возраст одновременно являются некорректными, должно быть возбуждено исключение, связанное с именем.

Sample Input 1:

```
user = User('Гвидо', 67)

print(user.get_name())
print(user.get_age())
```

Sample Output 1:

```
Гвидо
67
```

Sample Input 2:

```
user = User('Гвидо', 67)

user.set_name('Тимур')
user.set_age(30)

print(user.get_name())
print(user.get_age())
```

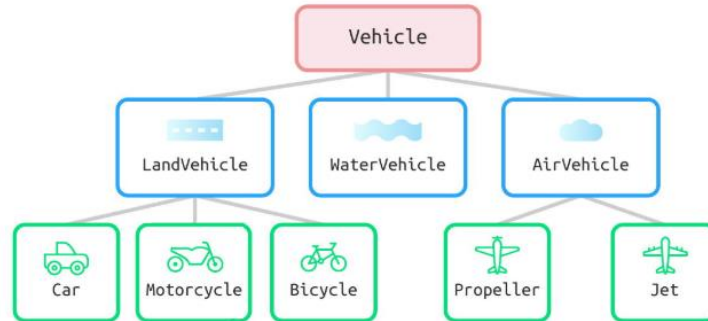
Sample Output 2:

```
Тимур
30
```

Наследование

Иерархия классов 🚗

С помощью наследования и приведенной ниже схемы постройте иерархию **пустых** классов, описывающих транспортные средства:



Sample Input 1:

```
print(issubclass(LandVehicle, Vehicle))
print(issubclass(WaterVehicle, Vehicle))
print(issubclass(AirVehicle, Vehicle))
```

Sample Output 1:

```
True
True
True
```

Sample Input 2:

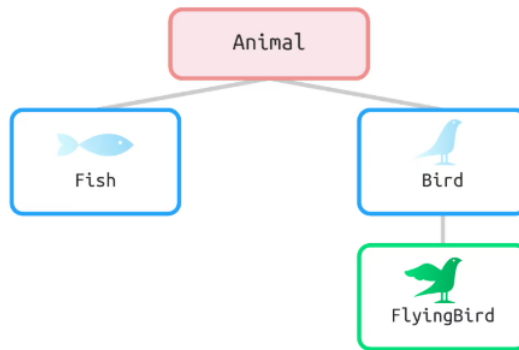
```
print(issubclass(Car, LandVehicle))
print(issubclass(Motorcycle, LandVehicle))
print(issubclass(Bicycle, LandVehicle))
```

Sample Output 2:

```
True
True
True
```

Иерархия классов 🐍

С помощью наследования и приведенной ниже схемы постройте иерархию классов, описывающих животных:



Класс `Animal` должен иметь два метода экземпляра:

- `sleep()` — пустой метод
- `eat()` — пустой метод

Класс `Fish` должен иметь один метод экземпляра:

- `swim()` — пустой метод

Класс `Bird` должен иметь один метод экземпляра:

- `lay_eggs()` — пустой метод

Класс `FlyingBird` должен иметь один метод экземпляра:

- `fly()` — пустой метод

Классы `Triangle` и `EquilateralTriangle`

Реализуйте класс `Triangle`, описывающий треугольник. При создании экземпляра класс должен принимать три аргумента в следующем порядке:

- `a` — длина стороны треугольника
- `b` — длина стороны треугольника
- `c` — длина стороны треугольника

Класс `Triangle` должен иметь один метод экземпляра:

- `perimeter()` — метод, возвращающий периметр треугольника

Также реализуйте класс `EquilateralTriangle`, наследника класса `Triangle`, описывающий равносторонний треугольник. При создании экземпляра класс должен принимать один аргумент:

- `side` — длина стороны треугольника

Примечание 1. Дополнительная проверка данных на корректность не требуется. Гарантируется, что реализованный класс используется только с корректными данными.

Примечание 2. Никаких ограничений касательно реализации классов нет, она может быть произвольной.

Sample Input 1:

```
print(issubclass(EquilateralTriangle, Triangle))
```

Sample Output 1:

```
True
```

Sample Input 2:

```
triangle1 = Triangle(3, 4, 5)
triangle2 = EquilateralTriangle(3)

print(triangle1.perimeter())
print(triangle2.perimeter())
```

Sample Output 2:

```
12
9
```

Класс LowerString

Реализуйте класс `LowerString`, наследника класса `str`, описывающий строку, которая во время создания автоматически переводится в нижний регистр. При создании экземпляра класс должен принимать один аргумент:

- `obj` — объект, определяющий начальное значение строки. Может быть не передан, в таком случае начальное значение считается пустой строкой

Примечание 1. Дополнительная проверка данных на корректность не требуется. Гарантируется, что реализованный класс используется только с корректными данными.

Примечание 2. Никаких ограничений касательно реализации класса `LowerString` нет, она может быть произвольной.

Sample Input 1:

```
s1 = LowerString('BEEGEEK')
s2 = LowerString('BeeGeek')

print(s1)
print(s2)
print(s1 == s2)
print(issubclass(LowerString, str))
```

Sample Output 1:

```
beegeek
beegeek
True
True
```

Класс TwoWayDict

Реализуйте класс `TwoWayDict`, наследника класса `UserDict`, описывающий двунаправленный словарь, в который при добавлении пары `ключ: значение` также добавляется и пара `значение: ключ`. Процесс создания экземпляра класса `TwoWayDict` должен совпадать с процессом создания экземпляра класса `UserDict`.

Примечание 1. Дополнительная проверка данных на корректность не требуется. Гарантируется, что реализованный класс используется только с корректными данными.

Примечание 2. Никаких ограничений касательно реализации класса `TwoWayDict` нет, она может быть произвольной.

Sample Input 1:

```
twowaydict = TwoWayDict({'apple': 1})

twowaydict['banana'] = 2

print(twowaydict['apple'])
print(twowaydict[1])
print(twowaydict['banana'])
print(twowaydict[2])
```

Sample Output 1:

```
1
apple
2
banana
```

Класс `AdvancedList`

Реализуйте класс `AdvancedList`, наследника класса `list`, описывающий список с дополнительным функционалом. Процесс создания экземпляра класса `AdvancedList` должен совпадать с процессом создания экземпляра класса `list`.

Класс `AdvancedList` должен иметь три метода экземпляра:

- `join()` — метод, объединяющий все элементы экземпляра класса `AdvancedList` в строку и возвращающий полученный результат. Метод должен принимать один строковый аргумент, по умолчанию равный пробелу, который является разделителем элементов списка в результирующей строке
- `map()` — метод, принимающий в качестве аргумента функцию `func` и применяющий ее к каждому элементу экземпляра класса `AdvancedList`. Метод должен изменять исходный экземпляр класса `AdvancedList`, а не возвращать новый
- `filter()` — метод, принимающий в качестве аргумента функцию `func` и удаляющий из экземпляра класса `AdvancedList` те элементы, для которых функция `func` вернула значение `False`. Метод должен изменять исходный экземпляр класса `AdvancedList`, а не возвращать новый

Примечание 1. Дополнительная проверка данных на корректность не требуется. Гарантируется, что реализованный класс используется только с корректными данными.

Примечание 2. Никаких ограничений касательно реализации класса `AdvancedList` нет, она может быть произвольной.

Sample Input 1:

```
advancedlist = AdvancedList([1, 2, 3, 4, 5])

print(advancedlist.join())
print(advancedlist.join('-'))
```

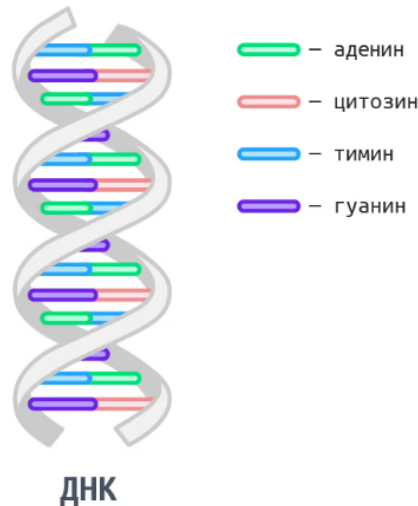
Sample Output 1:

```
1 2 3 4 5
1-2-3-4-5
```

абстрактные классы, модуль abc

Класс DNA

ДНК состоит из двух цепей, ориентированных азотистыми основаниями друг к другу. В ДНК встречается четыре вида азотистых оснований: аденин (A), гуанин (G), тимин (T) и цитозин (C). Азотистые основания одной из цепей соединены с азотистыми основаниями другой цепи водородными связями согласно принципу комплементарности: аденин (A) соединяется только с тимином (T), гуанин (G) — только с цитозином (C).



Реализуйте класс `DNA`, описывающий двухцепочную спираль ДНК. При создании экземпляра класс должен принимать один аргумент:

- `chain` — первая цепь ДНК, представляющая собой строку из символов `A`, `G`, `T` и `C` (азотистых оснований)

Экземпляр класса `DNA` должен иметь следующее неформальное строковое представление:

```
<азотистые основания первой цепи ДНК>
```

При передаче экземпляра класса `DNA` в функцию `len()` должно возвращаться количество азотистых оснований в одной из его цепей. При передаче экземпляра класса в функцию `reversed()` должен возвращаться итератор, элементами которого являются элементы переданного экземпляра класса `DNA`, расположенные в обратном порядке.

Помимо этого, экземпляр класса `DNA` должен быть итерируемым объектом, то есть позволять перебирать свои элементы, например, с помощью цикла `for`.

Также экземпляр класса `DNA` должен позволять получать значения своих элементов с помощью индексов, причем как положительных, так и отрицательных.

В случае с функцией `reversed()`, итерацией и доступу по индексам элементы экземпляра класса `DNA` должны быть представлены в виде кортежей из двух элементов, первым из которых является основание первой цепи ДНК по указанному индексу, вторым — азотистое основание второй цепи ДНК по указанному индексу. Азотистое основание второй цепи ДНК можно получить при помощи принципа комплементарности.

Вдобавок ко всему, экземпляр класса `DNA` должен поддерживать операцию проверки на принадлежность с помощью оператора `in`. В данном случае должно проверяться, входит ли азотистое основание в первую цепь ДНК.

Экземпляры класса `DNA` должны поддерживать между собой операции сравнения с помощью операторов `==` и `!=`. Две ДНК считаются равными, если их первые цепи совпадают.

Наконец, экземпляры класса `DNA` должны поддерживать между собой операцию сложения с помощью оператора `+`, результатом которой должен являться новый экземпляр класса `DNA`, первой цепью которого является конкатенация первых цепей исходных экземпляров класса `DNA`.

Примечание 1. Перед решением подумайте, какой абстрактный класс из модуля `collections.abc` будет удобен в качестве родительского.

Примечание 2. Если объект, с которым выполняется операция сравнения или сложения, некорректен, метод, реализующий эту операцию, должен вернуть константу `NotImplemented`.

Примечание 3. Никаких ограничений касательно реализации класса `DNA` нет, она может быть произвольной.

Sample Input 1:

```
dna = DNA('AGTC')

print(dna[0])
print(dna[1])
print(dna[2])
print(dna[3])
print(dna[-1])
print(dna[-2])
```

Sample Output 1:

```
('A', 'T')
('G', 'C')
('T', 'A')
('C', 'G')
('C', 'G')
('T', 'A')
```

Классы Average, Median и Harmonic

Вам доступны классы `Average`, `Median` и `Harmonic`, имеющие сходный интерфейс. Все три класса используются для обработки пользовательских оценок и оценок критиков некоторого медиаконтента по стобалльной шкале и вычисления средних значений этих оценок. Задачей класса `Average` является нахождение среднего арифметического пользовательских оценок или оценок критиков, классов `Median` и `Harmonic` — медианы и среднего гармонического соответственно.

Изучите приведенные классы, реализуйте абстрактный базовый класс `Middle` и постройте корректную схему наследования. При выполнении старайтесь избегать дублирования кода.

Примечание 1. Функционал классов `Average`, `Median` и `Harmonic` должен остаться прежним, необходимо лишь объединить их в иерархию, определив для них единый базовый абстрактный класс `Middle`.

Sample Input 1:

```
user_votes = [99, 90, 71, 1, 1, 100, 56, 60, 80]
expert_votes = [87, 90, 67, 70, 81, 85, 97, 79, 71]
average = Average(user_votes, expert_votes)

print(average.get_correct_user_votes())
print(average.get_correct_expert_votes())
print(average.get_average())
print(average.get_average(False))
```

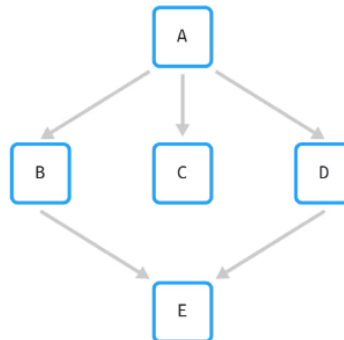
Sample Output 1:

```
[71, 56, 60, 80]
[87, 90, 67, 70, 81, 85, 79, 71]
66.75
78.75
```

Множественное наследование

Иерархия классов

С помощью наследования и приведенной ниже схемы постройте иерархию **пустых** классов:



Sample Input 1:

```
print(issubclass(E, B))
print(issubclass(E, C))
print(issubclass(E, D))
```

Sample Output 1:

```
True
False
True
```

Семья

С помощью множественного наследования постройте иерархию из приведенных ниже четырех классов. При решении старайтесь свести дублирование кода к минимуму.

1. Реализуйте класс `Father`, описывающий отца. При создании экземпляра класс должен принимать один аргумент:

- `mood` — настроение, по умолчанию равняется строке `neutral`

Экземпляр класса `Father` должен иметь один атрибут:

- `mood` — настроение

Класс `Father` должен иметь два метода экземпляра:

- `greet()` — метод, возвращающий строку `Hello!`
- `be_strict()` — метод, изменяющий значение атрибута `mood` на строку `strict`

2. Также реализуйте класс `Mother`, описывающий мать. При создании экземпляра класс должен принимать один аргумент:

- `mood` — настроение, по умолчанию равняется строке `neutral`

Экземпляр класса `Mother` должен иметь один атрибут:

- `mood` — настроение

Класс `Mother` должен иметь два метода экземпляра:

- `greet()` — метод, возвращающий строку `Hi, honey!`
- `be_kind()` — метод, изменяющий значение атрибута `mood` на строку `kind`

3. Помимо этого, реализуйте класс `Daughter`, описывающий дочь. При создании экземпляра класс должен принимать один аргумент:

- `mood` — настроение, по умолчанию равняется строке `neutral`

Экземпляр класса `Daughter` должен иметь один атрибут:

- `mood` — настроение

Класс `Daughter` должен иметь три метода экземпляра:

- `greet()` — метод, возвращающий строку `Hi, honey!`
- `be_kind()` — метод, изменяющий значение атрибута `mood` на строку `kind`
- `be_strict()` — метод, изменяющий значение атрибута `mood` на строку `strict`

4. Наконец, реализуйте класс `Son`, описывающий сына. При создании экземпляра класс должен принимать один аргумент:

- `mood` — настроение, по умолчанию равняется строке `neutral`

Экземпляр класса `Son` должен иметь один атрибут:

- `mood` — настроение

Класс `Son` должен иметь три метода экземпляра:

- `greet()` — метод, возвращающий строку `Hello!`
- `be_kind()` — метод, изменяющий значение атрибута `mood` на строку `kind`
- `be_strict()` — метод, изменяющий значение атрибута `mood` на строку `strict`

Примечание 1. Дополнительная проверка данных на корректность не требуется. Гарантируется, что реализованные классы используются только с корректными данными.

Примечание 2. Никаких ограничений касательно реализаций классов нет, они могут быть произвольными.

Sample Input 1:

```
father = Father()
mother = Mother()

print(father.mood)
print(mother.mood)
print(father.greet())
print(mother.greet())
```

Sample Output 1:

```
neutral
neutral
Hello!
Hi, honey!
```

Полиморфизм

Классы USADate и ItalianDate

1. Реализуйте класс `USADate`, описывающий дату в американском формате. При создании экземпляра класс должен принимать три аргумента в следующем порядке:

- `year` — год
- `month` — месяц
- `day` — день

Класс `USADate` должен иметь два метода экземпляра:

- `format()` — метод, который возвращает строку, представляющую собой дату в формате `MM-DD-YYYY`
- `iso_format()` — метод, который возвращает строку, представляющую собой дату в формате `YYYY-MM-DD`

2. Также реализуйте класс `ItalianDate`, описывающий дату в итальянском формате, конструктор которого принимает три аргумента:

- `year` — год
- `month` — месяц
- `day` — день

Класс `ItalianDate` должен иметь два метода экземпляра:

- `format()` — метод, который возвращает строку, представляющую собой дату в формате `DD/MM/YYYY`
- `iso_format()` — метод, который возвращает строку, представляющую собой дату в формате `YYYY-MM-DD`

Примечание 1. Дополнительная проверка данных на корректность не требуется. Гарантируется, что реализованные классы используются только с корректными данными.

Примечание 2. Никаких ограничений касательно реализаций классов нет, они могут быть произвольными.

Sample Input 1:

```
usadate = USADate(2023, 4, 6)

print(usadate.format())
print(usadate.iso_format())
```

Sample Output 1:

```
04-06-2023
2023-04-06
```

Классы LeftParagraph и RightParagraph

Будем называть словом любую последовательность из одной или более латинских букв.

1. Реализуйте класс `LeftParagraph`, описывающий абзац, выровненный по левому краю. При создании экземпляра класс должен принимать один аргумент:

- `length` — длина строки абзаца

Класс `LeftParagraph` должен иметь два метода экземпляра:

- `add()` — метод, принимающий в качестве аргумента слово или несколько слов, разделенных пробелом, и добавляющий их в текущий абзац. Если слово не помещается на текущей строке, оно переносится на следующую. Также метод должен автоматически добавлять один пробел после каждого добавленного слова (кроме последнего)
- `end()` — метод, печатающий текущий абзац, выровненный по левому краю. После вызова метода текущий абзац считается пустым, то есть начинается формирование нового

2. Также реализуйте класс `RightParagraph`, описывающий абзац, выровненный по правому краю. При создании экземпляра класс должен принимать один аргумент:

- `length` — длина строки абзаца

Класс `RightParagraph` должен иметь два метода экземпляра:

- `add()` — метод, принимающий в качестве аргумента слово или несколько слов, разделенных пробелом, и добавляющий их в текущий абзац. Если слово не помещается на текущей строке, оно переносится на следующую. Также метод должен автоматически добавлять один пробел после каждого добавленного слова (кроме последнего)
- `end()` — метод, печатающий текущий абзац, выровненный по правому краю с учетом длины строки. После вызова метода текущий абзац считается пустым, то есть начинается формирование нового

Примечание 1. Дополнительная проверка данных на корректность не требуется. Гарантируется, что реализованные классы используются только с корректными данными.

Примечание 2. Никаких ограничений касательно реализаций классов нет, они могут быть произвольными.

Sample Input 1:

```
leftparagraph = LeftParagraph(10)

leftparagraph.add('death')
leftparagraph.add('can have me')
leftparagraph.add('when it earns me')
leftparagraph.end()
```

Sample Output 1:

```
death can
have me
when it
earns me
```